

数学方向算法选讲

wsy

2026 年 8 月 4 日

目录

概率与期望

二项式定理与容斥原理

线性代数相关知识

概率论的研究对象

研究一个随机现象时，通常需要先明确三个对象：

- ▶ 样本空间 Ω ：所有可能出现的基本结果的集合；
- ▶ 事件域 $\mathcal{F}$ ：我们关心、并且允许讨论概率的事件集合；
- ▶ 概率函数 P ：对每一个事件 $A \in \mathcal{F}$ ，给出其发生的可能性大小。

三者合在一起构成概率空间 $(\Omega, \mathcal{F}, P)$ 。讨论概率时，必须先说明这个概率空间。

样本空间与随机事件

一个随机现象中不能再细分的结果称为样本点，所有样本点构成样本空间 Ω 。

随机事件是样本空间的一个子集，通常记为 $A, B, C, \dots$ 。若一次随机试验的结果为 ω ，则事件 A 发生当且仅当 $\omega \in A$ 。

例：掷一次骰子。

$$\Omega = \{1, 2, 3, 4, 5, 6\}$$

若 A 表示「点数大于 4」，则 $A = \{5, 6\}$ 。

事件的运算

事件本质上是集合，因此可以直接使用集合运算：

- ▶ $A \cup B$: A 或 B 发生，也常记作 $A + B$ ，称为和事件；
- ▶ $A \cap B$: A 与 B 同时发生，也常记作 AB ，称为积事件；
- ▶ $\bar{A}$: A 不发生，称为 A 的对立事件；
- ▶ $A - B$: A 发生但 B 不发生。

之后很多概率公式，本质上都是集合关系在概率函数下的翻译。

事件域

事件域 $\mathcal{F}$ 表示允许讨论概率的事件集合。有限样本空间中常取 $\mathcal{F} = 2^\Omega$ ，但一般情况下不一定需要、也不一定适合这么做。

为了保证常见事件运算仍然有意义， $\mathcal{F}$ 通常满足：

- ▶ $\emptyset \in \mathcal{F}$;
- ▶ 若 $A \in \mathcal{F}$ ，则 $\bar{A} \in \mathcal{F}$;
- ▶ 若 $A_1, A_2, \dots \in \mathcal{F}$ ，则 $\bigcup_{i \geq 1} A_i \in \mathcal{F}$ 。

也就是说，事件域对补运算和可数并封闭。

概率的定义

古典概型：若样本空间有限，并且每个样本点等可能出现，则

$$P(A) = \frac{\#(A)}{\#(\Omega)}.$$

概率的定义

公理化定义：概率函数是一个映射

$$P: \mathcal{F} \rightarrow [0, 1],$$

并满足：

- ▶ 规范性： $P(\Omega) = 1$;
- ▶ 可数可加性：若 $A_1, A_2, \dots$ 两两不交，则
$$P\left(\bigcup_{i \geq 1} A_i\right) = \sum_{i \geq 1} P(A_i)。$$

概率函数的基本性质

对任意事件 $A, B \in \mathcal{F}$, 有:

- ▶ $P(\emptyset) = 0$, $P(\bar{A}) = 1 - P(A)$;
- ▶ 单调性: 若 $A \subset B$, 则 $P(A) \leq P(B)$;
- ▶ 容斥原理: $P(A + B) = P(A) + P(B) - P(AB)$;
- ▶ 差事件: $P(A - B) = P(A) - P(AB)$ 。

这些性质在算法分析中会反复出现, 例如估计成功概率、失败概率或多个事件同时发生的概率。

条件概率

当已知事件 A 已经发生时，事件 B 发生的概率可能会发生变化，这称为条件概率，记作 $P(B|A)$ 。

若 $P(A) > 0$ ，定义

$$P(B|A) = \frac{P(AB)}{P(A)}.$$

直观上，就是把样本空间缩小到 A ，再看其中有多少比例同时落在 B 中。

由定义可得乘法公式：

$$P(AB) = P(A)P(B|A).$$

全概率公式

若事件 $A_1, A_2, \dots, A_n$ 两两不交，并且

$$A_1 + A_2 + \dots + A_n = \Omega,$$

则称它们构成样本空间的一个划分。

对任意事件 B ，有

$$P(B) = \sum_{i=1}^n P(A_i)P(B|A_i).$$

含义：把 B 按照「它是在哪一种情况 A_i 下发生的」拆开分别计算。

Bayes 公式

全概率公式用于从原因推出结果；Bayes 公式用于从结果反推原因。

若 $A_1, A_2, \dots, A_n$ 构成样本空间的一个划分，且 $P(B) > 0$ ，则

$$P(A_i|B) = \frac{P(A_i)P(B|A_i)}{\sum_{j=1}^n P(A_j)P(B|A_j)}.$$

其中 $P(A_i)$ 常称为先验概率， $P(A_i|B)$ 常称为后验概率。

事件的独立性

若事件 A, B 满足

$$P(AB) = P(A)P(B),$$

则称 A 与 B 独立。

当 $P(A) > 0$ 时，这等价于

$$P(B|A) = P(B).$$

也就是说，知道 A 发生并不会改变我们对 B 发生概率的判断。

多个事件的独立性

多个事件独立要求任意子集都满足概率可以拆成乘积。对于 $A_1, A_2, \dots, A_n$ ，任取若干个不同下标，都应有

$$P(A_{i_1} A_{i_2} \cdots A_{i_r}) = \prod_{k=1}^r P(A_{i_k}).$$

多个事件的独立性

注意：两两独立不能推出整体独立。

反例中可以出现

$$P(AB) = P(A)P(B), \quad P(BC) = P(B)P(C), \quad P(CA) = P(C)P(A),$$

但 $P(ABC) \neq P(A)P(B)P(C)$ 。

随机变量的期望

期望描述随机变量的平均取值。

若离散型随机变量 X 满足

$$P(X = x_i) = p_i,$$

且 $\sum_i x_i p_i$ 绝对收敛, 则

$$E[X] = \sum_i x_i p_i.$$

随机变量的期望

若连续型随机变量 X 的概率密度为 $f(x)$ ，且积分绝对收敛，则

$$E[X] = \int_{-\infty}^{+\infty} xf(x) dx.$$

期望不一定存在，不能只看和式或积分是否条件收敛。

期望的线性性

期望最重要的性质是线性性。若对应期望存在，则

$$E[aX + b] = aE[X] + b,$$

$$E[X + Y] = E[X] + E[Y].$$

期望的线性性

推广到多个随机变量：

$$E\left[\sum_{i=1}^n X_i\right] = \sum_{i=1}^n E[X_i].$$

注意：线性性不要求这些随机变量相互独立。

乘积的期望

若随机变量 X, Y 相互独立，并且期望存在，则

$$E[XY] = E[X]E[Y].$$

这个结论常用于分析多次独立随机试验的乘积型贡献。

但要小心：

- ▶ 独立性是推出该式的常用充分条件；
- ▶ 不独立时该式可能成立，也可能不成立；
- ▶ 一般情况下不能把 $E[XY]$ 随意拆成 $E[X]E[Y]$ 。

示性函数

对事件 A ，定义示性函数

$$I_A(\omega) = \begin{cases} 1, & \omega \in A, \\ 0, & \omega \notin A. \end{cases}$$

则

$$E[I_A] = P(A).$$

因此可以把概率问题转化为期望问题，也可以把随机变量拆成若干个事件贡献之和。

示性函数的例子

设序列 $a_1, a_2, \dots, a_n$ 中, a_k 以概率 p_k 取值 k , 以概率 $1 - p_k$ 取值 0。

令 I_k 表示事件 $a_k = k$ 的示性函数, 则

$$a_k = kI_k, \quad E[I_k] = p_k.$$

示性函数的例子

因此

$$E \left[\sum_{k=1}^n a_k \right] = E \left[\sum_{k=1}^n k I_k \right] = \sum_{k=1}^n k p_k.$$

这里完全不需要枚举总和的每一种可能取值。

关于概率和期望

很多题让你求一个事件的概率，本质上其实是让你算合法的方案数量，然后除掉总的方案数量即可。

很多题让你求一个值的期望，本质上是让你算所有可能情况中，这个值的和。

因此不要被表面上的期望和概率吓到。

但是当然，很多题根本不能这么做，需要用到一些概率和期望的知识。这也是真的。

P1654 OSU!

有 n 次操作，第 i 次以概率 p_i 成功，成功记为 1，失败记为 0。

最终得到一个 01 串，每个极长连续 1 段，若长度为 x ，贡献 x^3 分。求总分的期望。

$n \leq 10^5$ 。

设扫完前 i 位后的后缀连续 1 长度为随机变量 L_i , 前缀得分为随机变量 S_i 。

若第 i 位成功, 且原本 $L_{i-1} = x$, 则最后一段长度从 x 变成 $x+1$, 得分增量为

$$(x+1)^3 - x^3 = 3x^2 + 3x + 1.$$

因此只需要维护

$$a_i = E[L_i], \quad b_i = E[L_i^2], \quad f_i = E[S_i].$$

第 i 位失败时 $L_i = 0$; 成功时 $L_i = L_{i-1} + 1$ 。于是

$$a_i = p_i(a_{i-1} + 1),$$

$$b_i = p_i(b_{i-1} + 2a_{i-1} + 1).$$

由增量贡献可得

$$f_i = f_{i-1} + p_i(3b_{i-1} + 3a_{i-1} + 1).$$

初始 $a_0 = b_0 = f_0 = 0$, 顺序扫描这些期望量即可。

CF1942G Bessie and Cards

有 a 张“抽 0 张”卡， b 张“抽 1 张”卡， c 张“抽 2 张”卡，以及 5 张特殊卡，所有卡随机排列。

初始抽取牌堆顶端 5 张牌。之后可以打出手牌中的“抽 x 张”卡，继续从牌堆顶抽 x 张牌；每张卡至多使用一次，特殊卡不能打出。

若最终能抽到全部 5 张特殊卡，则获胜。求获胜概率，对 998244353 取模。

$t \leq 10^4$ ，所有测试中 a, b, c 的总和分别不超过 2×10^5 。

CF1942G Bessie and Cards

“抽 1 张”卡不影响过程：抽到后立刻打出，只是把后面一张牌继续翻开。

因此先删去所有“抽 1 张”卡。令余额表示还能继续翻开的牌数：

- ▶ 初始余额为 5；
- ▶ “抽 2 张”卡使余额 $+1$ ；
- ▶ “抽 0 张”卡和特殊卡都使余额 -1 。

过程会在余额第一次变为 0 时停止。获胜当且仅当停止前已经见到全部 5 张特殊卡。

CF1942G Bessie and Cards

计算失败概率。设停止时，之前一共出现了 i 张“抽 2 张”卡。

停止牌一定是一张 -1 牌；在它之前有 i 张 $+1$ 牌和 $i+4$ 张 -1 牌，并且所有前缀余额都为正。

由反射原理，这样的相对顺序数为

$$N_i = \binom{2i+4}{i} - \binom{2i+4}{i-1}.$$

这一步本质上是在数一条从初始高度 5 出发、首次回到 0 的路径。

CF1942G Bessie and Cards

令所有 -1 牌共有 $a+5$ 张，其中 5 张是特殊卡。

若在停止前后共见到的 $i+5$ 张 -1 牌中没有包含全部特殊卡，则失败。对应选择数为

$$M_i = \sum_{k=0}^4 \binom{i+5}{k} \binom{a-i}{5-k}.$$

剩余牌的排列方式为

$$R_i = \binom{a+c-2i}{c-i}.$$

CF1942G Bessie and Cards

对每个 i , 失败贡献为

$$N_i M_i R_i.$$

总状态数为

$$\binom{a+c+5}{c} \binom{a+5}{5}.$$

CF1942G Bessie and Cards

因此

$$P_{\text{lose}} = \frac{\sum_{i=0}^c N_i M_i R_i}{\binom{a+c+5}{c} \binom{a+5}{5}}.$$

最后

$$P_{\text{win}} = 1 - P_{\text{lose}}.$$

最后把所有失败情况汇总，用总方案数归一化即可。

CF235D Graph Game

给定一张 n 个点、 n 条边的连通无向图。

定义递归过程 $solve(t)$:

- ▶ 令 $totalCost \leftarrow totalCost + |t|$;
- ▶ 在当前连通图 t 中等概率随机选择一个点 x ;
- ▶ 删除 x , 对剩下的每个连通分量递归执行。

求最终 $totalCost$ 的期望。

$3 \leq n \leq 3000$, 答案允许 10^{-6} 误差。

CF235D Graph Game

给每个点独立随机一个优先级。每个连通分量中，最先被删除的点就是优先级最小的点。

考虑一次删除点 x 时产生的代价，它等于删除所有优先级小于 x 的点后， x 所在连通块大小。

因此

$$E[\text{totalCost}] = \sum_x \sum_y \Pr[y \text{ 在 } x \text{ 被删时仍与 } x \text{ 连通}].$$

如果 x, y 之间只有一条简单路径，设路径上点数为 d ，则要求 x 是这 d 个点中优先级最小的点，概率为 $\frac{1}{d}$ 。

CF235D Graph Game

因为图有 n 个点、 n 条边且连通，所以它是一张基环树。先找出唯一的环，并把每个非环点挂到某个环点为根的树上。

对有序点对 (x, y) :

- ▶ 若 x, y 在同一棵挂树中，只存在唯一树上路径，贡献为 $\frac{1}{d}$;
- ▶ 若 x, y 属于不同环根，则有两条绕环路径，做容斥。

这一步把随机过程完全转化成了图上点对贡献求和。

CF235D Graph Game

设环长为 C , x, y 到各自环根的距离为 h_x, h_y 。两环根在环上的两段距离分别为 s 和 $C - s$ 。

两条简单路径的点数为

$$d_1 = h_x + h_y + s + 1, \quad d_2 = h_x + h_y + (C - s) + 1.$$

两条路径并集的点数为

$$d_3 = h_x + h_y + C.$$

CF235D Graph Game

两条路径至少有一条保持连通即可。

用容斥计算该有序点对的贡献：

$$\frac{1}{d_1} + \frac{1}{d_2} - \frac{1}{d_3}.$$

最后对所有有序点对求和即可。

AT arc113 F Social Distance

给定

$$0 = X_0 < X_1 < \cdots < X_N.$$

第 i 个人会在区间 $[X_{i-1}, X_i]$ 中均匀随机选择一个实数坐标。

求所有人之间最小距离的期望值，对 998244353 取模。

$2 \leq N \leq 20$, $X_N \leq 10^6$ 。

AT arc113 F Social Distance

设答案随机变量为 D 。因为 $D \geq 0$ ，有尾积分公式

$$E[D] = \int_0^{+\infty} \Pr[D \geq z] dz.$$

固定 z ，设第 i 个人的位置为 A_i 。由于区间按顺序排列，

$$D \geq z \iff A_{i+1} - A_i \geq z \quad (1 \leq i < N).$$

AT arc113 F Social Distance

令

$$B_i = A_i - (i-1)z,$$

则问题变成： B_i 在区间

$$[X_{i-1} - (i-1)z, X_i - (i-1)z]$$

中均匀选取，求 $B_1 \leq B_2 \leq \dots \leq B_N$ 的概率。

AT arc113 F Social Distance

固定 z 后，把所有新区间端点离散化为

$$p_1 < p_2 < \cdots < p_m.$$

设 $f_{i,j}$ 表示前 i 个数已经满足单调，且 $B_i \in [p_j, p_{j+1})$ 的概率。

AT arc113 F Social Distance

枚举最后一段连续落在同一个小区间 $[p_j, p_{j+1})$ 的位置 $k, k+1, \dots, i$:

$$f_{i,j} = \sum_k \frac{(p_{j+1} - p_j)^{i-k+1}}{\prod_{l=k}^i (X_l - X_{l-1})} \frac{1}{(i-k+1)!} \sum_{q < j} f_{k-1,q}.$$

其中要求第 $k, \dots, i$ 个新区间都包含 $[p_j, p_{j+1})$ 。前缀和优化后，固定 z 可 $O(N^3)$ 求出 $\Pr[D \geq z]$ 。

AT arc113 F Social Distance

难点在于 z 是连续变量。

所有端点都是一次函数：

$$X_i - iz.$$

只有当两个端点大小关系交换时，离散化顺序才会改变。这样的临界点只有 $O(N^2)$ 个。

将 z 轴按这些临界点切成若干段。在每一段内：

- ▶ 端点相对顺序不变；
- ▶ 每个小区间长度都是关于 z 的一次多项式；
- ▶ DP 中的所有值都是关于 z 的多项式。

AT arc113 F Social Distance

在每一段上：

- ▶ 用多项式表示 DP 值；
- ▶ 得到 $\Pr[D \geq z]$ 关于 z 的多项式；
- ▶ 对该多项式精确积分。

把所有段的积分结果相加，即为 $E[D]$ 。

把所有段上的积分结果相加，即为最终期望。

AT agc032 F One Third

一个圆形披萨被随机切 N 刀，每刀从圆心切到圆周上一个均匀随机角度。

于是披萨被分成 N 块。可以选择连续若干块，设总量为 x ，使得

$$\left|x - \frac{1}{3}\right|$$

尽可能小。

求这个最小值的期望，对 $10^9 + 7$ 取模。

$2 \leq N \leq 10^6$ 。

AT agc032 F One Third

对每一刀的位置 p ，额外标出

$$p + \frac{1}{3}, \quad p - \frac{1}{3}$$

这两个位置，并把三类位置染成三种颜色。

选择一段连续披萨接近 $\frac{1}{3}$ ，等价于在圆上找距离最近的一对异色标记点。

固定第一刀产生的三条线，它们把圆分成三段。由对称性，只看其中长度为 $\frac{1}{3}$ 的一段：

- ▶ 内部有 $N - 1$ 个随机点；
- ▶ 每个点独立随机染三色之一；
- ▶ 两个端点颜色固定且不同。

AT agc032 F One Third

暂时忽略颜色。长度为 1 的线段上随机撒 $n-1$ 个点，切成 n 段。

最短段长度 $L_{\min}$ 满足

$$\Pr[L_{\min} \geq x] = (1 - nx)^{n-1}.$$

因此

$$E[L_{\min}] = \int_0^{1/n} (1 - nx)^{n-1} dx = \frac{1}{n^2}.$$

AT agc032 F One Third

更一般地，设 L_k 为第 k 短的段长。

可以用同样的尾积分方法求出

$$E[L_k] = \frac{1}{n} \sum_{j=1}^k \frac{1}{n-j+1}.$$

后面只需要知道：第 k 短线段的期望有一个可以 $O(1)$ 增量维护的表达式。

AT agc032 F One Third

回到颜色限制。若按长度从小到大看线段，第一个两端异色的线段是第 k 短，则前 $k - 1$ 条两端同色，第 k 条两端异色。

其概率为

$$\frac{1}{3^{k-1}} - \frac{1}{3^k}.$$

AT agc032 F One Third

原线段长度为 $\frac{1}{3}$ ，所以最终答案为

$$\frac{1}{3} \sum_{k=1}^N \left(\frac{1}{3^{k-1}} - \frac{1}{3^k} \right) E[L_k] = \frac{1}{N} \sum_{k=1}^N \frac{1}{3^k(N-k+1)}.$$

这个求和式已经把“长度顺序”和“颜色限制”两部分合并起来。

AT agc045 D Lamps and Buttons

有 N 盏灯和 N 个按钮，初始只有 $1, 2, \dots, A$ 号灯点亮。

Ringo 随机生成一个排列 $p_1, p_2, \dots, p_N$ ，Snuke 不知道它。按下按钮 i 会翻转灯 p_i 的状态。

每次 Snuke 只能选择当前点亮的灯对应的按钮来按，并且总能观察到所有灯的当前状态。目标是让所有灯都点亮。

求最优策略下胜率 w ，输出 $w \cdot N! \bmod (10^9 + 7)$ 。

$2 \leq N \leq 10^7$, $1 \leq A \leq \min(N - 1, 5000)$ 。

AT agc045 D Lamps and Buttons

将排列看成函数图：从 i 向 p_i 连边，于是所有点被分成若干置换环。

若一个环里有一个已经点亮且尚未确定出边的点，就可以尝试沿着这个环继续探索。

关键结论：

- ▶ 如果某个环完全不含初始点亮的点，则永远无法进入这个环；
- ▶ 如果操作到自环 $p_i = i$ ，会把当前亮着的灯直接熄灭，必败；
- ▶ 最优策略可看作按编号从小到大探索当前可探索的环。

因此问题转为统计哪些排列在这种探索中会成功。

AT agc045 D Lamps and Buttons

设 t 为区间 $[1, A]$ 中第一个满足 $p_t = t$ 的位置；若不存在，则令 $t = A + 1$ 。

想在遇到第一个自环前获胜，需要满足：

$$\forall i > A, \quad i \text{ 所在的环中存在某个 } j < t.$$

还要求 $1, 2, \dots, t-1$ 中没有自环。这个条件用容斥处理：枚举其中被强制为自环的个数 j ，系数为

$$(-1)^j \binom{t-1}{j}.$$

于是剩下只需数一类带限制的置换环分解。

AT agc045 D Lamps and Buttons

容斥后令

$$a = t - 1 - j, \quad b = N - A, \quad c = A - t.$$

需要统计大小为 $a + b + c$ 的排列，使得中间 b 个点所在的每个环都至少包含一个前 a 个点。

按点插入置换环：

- ▶ 先排列前 a 个点；
- ▶ 将 b 个必须被覆盖的点插入已有环中；
- ▶ 最后 c 个点可以自成环，也可以插入已有环。

AT agc045 D Lamps and Buttons

可得该数量为

$$(a + b + c)! \cdot \frac{a}{a + b}.$$

将它乘上前一页的容斥系数

$$(-1)^j \binom{t-1}{j}$$

后，对所有 t, j 求和。

最后对所有可能的 t, j 求和，得到成功排列的总数。

目录

概率与期望

二项式定理与容斥原理

线性代数相关知识

二项式定理

组合数： $\binom{n}{m}$ 表示从 n 个元素中选择 m 个的方案数。

二项式定理：

$$(x + y)^n = \sum_{i=0}^n \binom{n}{i} x^i y^{n-i}$$

证明可以对 n 进行归纳，或者直接理解也不难。

广义二项式定理：

$$(x_1 + \cdots + x_t)^n = \sum_{n_1 + \cdots + n_t = n} \binom{n}{n_1, \dots, n_t} x_1^{n_1} \cdots x_t^{n_t}$$

一些基础组合数恒等式

$$\binom{a}{b} \binom{b}{c} = \binom{a}{c} \binom{a-c}{b-c}$$

$$[n=0] = \sum_{i=0}^n \binom{n}{i} (-1)^i$$

二项式反演

第一种常见形式是：

$$f(n) = \sum_{i=0}^n (-1)^i \binom{n}{i} g(i) \iff g(n) = \sum_{i=0}^n (-1)^i \binom{n}{i} f(i)$$

证明如下：

$$\begin{aligned} \sum_{i=0}^n (-1)^i \binom{n}{i} f(i) &= \sum_{i=0}^n (-1)^i \binom{n}{i} \sum_{j=0}^i (-1)^j \binom{i}{j} g(j) \\ &= \sum_{j=0}^n g(j) \binom{n}{j} \sum_{i=j}^n (-1)^{i-j} \binom{n-j}{i-j} = \sum_{j=0}^n g(j) \binom{n}{j} [n-j=0] = g(n) \end{aligned}$$

二项式反演

第二种形式是：

$$f(n) = \sum_{i=n}^N (-1)^i \binom{i}{n} g(i) \iff g(n) = \sum_{i=n}^N (-1)^i \binom{i}{n} f(i)$$

证明也是类似的：

$$\begin{aligned} \sum_{i=n}^N (-1)^i \binom{i}{n} f(i) &= \sum_{i=n}^N (-1)^i \binom{i}{n} \sum_{j=i}^N \binom{j}{i} g(j) \\ &= \sum_{j=n}^N g(j) (-1)^{j-n} \binom{j}{n} \sum_{i=n}^j (-1)^{i-n} \binom{j-n}{i-n} \\ &= \sum_{j=n}^N g(j) (-1)^{j-n} \binom{j}{n} [j-n=0] = g(n) \end{aligned}$$

基础容斥原理

我们所说的容斥原理，最基础的版本的本质其实就是二项式定理。

例如题目让我们求所有元素都合法的方案数，容斥原理通常会说我们钦定了 k 个元素不合法，去求方案数。

本质上，我们令 $f(n)$ 表示恰好 n 个元素不合法的方案数， $g(n)$ 表示钦定了 n 个元素不合法的方案数。那么首先有 g 关于 f 的递推式满足上面的关系，因此我们可以用 g 来反过来求出 f 。这也证明了容斥原理的正确性。

至少与恰好

在计数题中，所谓“至少”经常不是朴素地限制数量至少为 k 。

更常见的是：

- ▶ 钦定了 k 个性质必须满足；
- ▶ 剩下的性质不作限制。

若 $f(k)$ 表示钦定 k 个性质满足的方案数， $g(k)$ 表示恰好 k 个性质满足的方案数，则

$$f(k) = \sum_{i=k}^N \binom{i}{k} g(i).$$

反演得到

$$g(k) = \sum_{i=k}^N (-1)^{i-k} \binom{i}{k} f(i).$$

P10596 BZOJ2839 集合计数

一个有 N 个元素的集合有 2^N 个不同子集。现在要从这些子集中选出若干个集合，至少选一个，使得它们的交集大小恰好为 K 。

求方案数，对 $10^9 + 7$ 取模。

$$1 \leq N \leq 10^6, \quad 0 \leq K \leq N.$$

P10596 BZOJ2839 集合计数

仍然是钦定。

令 $f(i)$ 表示钦定 i 个元素在交集中，其余元素不作限制的方案数。

令 $g(i)$ 表示交集大小恰好为 i 的方案数，答案就是 $g(K)$ 。

固定这 i 个元素后，每个被选的集合都必须包含它们；剩下 $N - i$ 个元素可以任意选择，所以可用的集合数为

$$2^{N-i}.$$

P10596 BZOJ2839 集合计数

固定钦定元素后，可用集合一共有 2^{N-i} 个。

从这些集合中选择一个非空集合族，方案数为

$$2^{2^{N-i}} - 1$$

再乘上钦定元素集合的选择方式，可得

$$f(i) = \binom{N}{i} (2^{2^{N-i}} - 1).$$

若实际交集大小为 j ，那么选哪 i 个钦定元素都可以，所以这类方案会在 $f(i)$ 中被统计 $\binom{j}{i}$ 次。

P10596 BZOJ2839 集合计数

所以有

$$f(i) = \sum_{j=i}^N \binom{j}{i} g(j).$$

这正是“至少”和“恰好”的转化，于是

$$g(K) = \sum_{i=K}^N (-1)^{i-K} \binom{i}{K} f(i).$$

代入 $f(i)$ 得

$$g(K) = \sum_{i=K}^N (-1)^{i-K} \binom{i}{K} \binom{N}{i} (2^{2^{N-i}} - 1).$$

P10596 BZOJ2839 集合计数

剩下就是理解这个式子中每一项的含义。

模数 $M = 10^9 + 7$ 是质数。计算

$$2^{2^{N-i}} \bmod M$$

时，指数可以先化成

$$2^{N-i} \bmod (M-1),$$

再代入原式。

P10597 BZOJ4665 小 w 的喜糖

有 n 块糖发给 n 个人，每块糖有一个种类。第 i 个人原来拿到的糖的种类为 A_i 。

现在这些人互相交换手中的糖，求有多少种方案使得每个人拿到的糖的种类都与原来不同。

同一种类的糖不可区分；两个方案不同当且仅当存在某个人拿到的糖的种类不同。

$$1 \leq A_i \leq n \leq 2000,$$

答案对 $10^9 + 9$ 取模。

P10597 BZOJ4665 小 w 的喜糖

直接数每个人都不同并不好做。令坏事件为：

E_x ： x 拿到的糖的种类与原来相同。

我们仍然先算钦定若干坏事件发生的方案数，再容斥掉。

设第 i 种糖果一共有 c_i 块。先暂时把同种糖果也看成互不相同，最后再除以

$$\prod_i c_i!$$

即可。

P10597 BZOJ4665 小 w 的喜糖

设 $f_{i,j}$ 表示只考虑前 i 种糖果时，已经钦定了 j 个人拿到原种类糖，并且这些钦定位置已经分配好具体糖果的方案数。

转移到第 i 种糖果时，枚举这类糖中钦定了 k 个人。

这 k 个人必须从原本拿着第 i 种糖的 c_i 个人中选择，具体糖果也要有序分给他们，因此系数为

$$\binom{c_i}{k} A_{c_i}^k.$$

这里 $A_n^k = n(n-1) \cdots (n-k+1)$ 。

P10597 BZOJ4665 小 w 的喜糖

转移方程为

$$f_{i,j} = \sum_{k=0}^{\min(c_i,j)} f_{i-1,j-k} \binom{c_i}{k} A_{c_i}^k.$$

初始

$$f_{0,0} = 1.$$

处理完所有出现过的糖果种类后， $f_{m,j}$ 表示钦定 j 个坏事件发生，并已经给这些人分好糖的方案数。

P10597 BZOJ4665 小 w 的喜糖

如果已经钦定了 j 个人拿到原种类糖，并分好了这些糖，那么剩下的 $n - j$ 块互不相同的糖可以任意分给剩下的人。

这一步有 $(n - j)!$ 种方案。

于是按坏事件容斥，互不相同糖果下的答案为

$$\sum_{j=0}^n (-1)^j f_{m,j} (n - j)!.$$

最后除以

$$\prod_i c_i!$$

去掉同种糖果之间的标号。

P10597 BZOJ4665 小 w 的喜糖

最终答案为

$$\left(\sum_{j=0}^n (-1)^j f_{m,j} (n-j)! \right) \prod_i (c_i!)^{-1}.$$

这就是“先按种类统计钦定坏事件，再整体容斥”的完整表达式。

P10982 Connected Graph

求 n 个结点的有标号无向连通图个数，对 1004535809 取模。

$$1 \leq n \leq 1000.$$

P10982 Connected Graph

先看所有有标号无向图的数量。任意两点之间的边可以选或不选，所以

$$f(n) = 2^{\frac{n(n-1)}{2}}.$$

设 $g(n)$ 表示 n 个点的有标号无向连通图数量，答案就是 $g(n)$ 。

考虑容斥式地写成：

$$g(n) = f(n) - n \text{ 个点的不连通图数量}.$$

对于一张不连通图，1 号点一定属于某个连通块。

枚举 1 号点所在连通块大小为 i ，其中 $1 \leq i < n$ 。这样每张不连通图会被唯一地统计一次。

P10982 Connected Graph

若 1 号点所在连通块大小为 i :

- ▶ 从剩下 $n - 1$ 个点中选择 $i - 1$ 个加入这个连通块;
- ▶ 这 i 个点内部必须连通, 有 $g(i)$ 种;
- ▶ 剩下 $n - i$ 个点之间可以任意连边, 有 $f(n - i)$ 种;
- ▶ 两部分之间不能有边。

因此

$$g(n) = f(n) - \sum_{i=1}^{n-1} \binom{n-1}{i-1} g(i) f(n-i).$$

P10982 Connected Graph

初始

$$f(0) = 1, \quad g(1) = 1.$$

从生成函数角度看，这就是把所有有标号图按连通块分解：

$$F(x) = \exp G(x),$$

所以

$$G(x) = \ln F(x).$$

上面的递推就是多项式 $\ln$ 的朴素 $\Theta(n^2)$ 写法。

卡特兰数

卡特兰数是一类非常常见的计数数列，通常记作 C_n ：

$$C_n = \frac{1}{n+1} \binom{2n}{n}.$$

前几项为

$$1, 1, 2, 5, 14, 42, 132, \dots$$

它经常出现于一类“前缀不能越界”的计数问题中。

典型特征是：总共有两类操作，各做 n 次，并且任意前缀中第一类操作的次数都不少于第二类操作。

卡特兰数

一个最经典的模型是合法括号序列。

长度为 $2n$ 的括号序列中，有 n 个左括号和 n 个右括号。要求任意前缀中左括号数量不少于右括号数量。

这样的序列数量就是

$$C_n.$$

例如 $n = 3$ 时有

$$((())), (())(), (())(), ()(()), ()()()$$

共 5 种。

卡特兰数

另一个等价模型是网格路径。

从 $(0, 0)$ 走到 (n, n) ，每次只能向右或向上走一步，并且任意前缀中向右步数不少于向上步数。

也就是说，路径始终不越过直线 $y = x$ 。若把向右看成左括号、向上看成右括号，就与合法括号序列完全等价。

总路径数是

$$\binom{2n}{n}.$$

不合法路径可以用反射原理数出来，数量为

$$\binom{2n}{n-1}.$$

卡特兰数

因此

$$C_n = \binom{2n}{n} - \binom{2n}{n-1} = \frac{1}{n+1} \binom{2n}{n}.$$

这也是卡特兰数最常用的计算式。

常见等价问题还包括：

- ▶ $n+1$ 个叶子的满二叉树形态数；
- ▶ n 个数依次入栈时，可能的出栈序列数量；
- ▶ 凸 $(n+2)$ 边形的三角剖分数；
- ▶ n 对点互不相交连线的方案数。

子集反演

子集反演是与二项式反演类似的东西，只不过反演的并不是两个一般的数列，而是对一个集合的所有子集进行。具体如下：

$$g(S) = \sum_{T \subseteq S} f(T) \iff f(S) = \sum_{T \subseteq S} (-1)^{|S|-|T|} g(T)$$

之前的容斥我们似乎只关心个数，而子集容斥则是针对每个元素互不相同的情况。

P3349 [ZJOI2016] 小星星

给定一棵 n 个点的树 T ，以及一张 n 个点的无向图 G 。

要给树上每个点 u 分配一个编号 x_u ，使得 $x_1, x_2, \dots, x_n$ 是 1 到 n 的一个排列。

同时要求：若树上有边 (u, v) ，则图中必须有边 (x_u, x_v) 。

求合法编号方案数。

$$n \leq 17.$$

P3349 [ZJOI2016] 小星星

直接做树形 DP 时，一个自然状态是

$$f_{u,j,S}$$

表示 u 映射到 j ，并且 u 子树内使用的编号集合为 S 的方案数。

这能保证最后所有编号互不相同。

但合并儿子时要关心不同子树分别使用了哪些编号，状态之间会变得很笨重。

这提示我们：真正麻烦的不是树上边关系，而是“所有编号互不相同”这个全局限制。

P3349 [ZJOI2016] 小星星

关键问题是：为什么需要记录 S ？

因为题目要求编号是一个排列，也就是每个编号恰好使用一次。

如果暂时去掉这个限制，只要求每个点的编号都属于某个集合 S ，那么就不用记录子树内具体用了哪些编号。

设 $F(S)$ 表示所有树点都映射到 S 中，并且边关系合法的方案数，允许多个树点映射到同一个编号。

最后再用子集容斥，强制所有编号都至少出现一次。

P3349 [ZJOI2016] 小星星

固定一个编号集合 S ，根定树。设

$$dp_{u,j}$$

表示在 u 的子树内， u 映射到图上点 j 的方案数，其中要求 $j \in S$ 。

若 $j \notin S$ ，则 $dp_{u,j} = 0$ 。

对于 u 的每个儿子 v ， v 的编号 k 必须满足 $k \in S$ 且 (j, k) 是图中的边。

因此

$$dp_{u,j} = \prod_{v \in \text{son}(u)} \sum_{\substack{k \in S \\ (j,k) \in E(G)}} dp_{v,k}.$$

P3349 [ZJOI2016] 小星星

固定 S 后，树形 DP 的总方案数为

$$F(S) = \sum_{j \in S} dp_{root,j}.$$

现在考虑答案。因为树和图都有 n 个点，所以“每个编号至少出现一次”等价于“编号是一个排列”。

对所有编号集合做子集容斥：

$$ans = \sum_{S \subseteq [n]} (-1)^{n-|S|} F(S).$$

这就是把“编号可以重复”的方案，反演成“所有编号都出现”的方案。

P3349 [ZJOI2016] 小星星

从大小角度看，就是：

$$ans = \sum_{|S|=n} F(S) - \sum_{|S|=n-1} F(S) + \sum_{|S|=n-2} F(S) - \dots$$

但注意真正枚举的是每个具体子集 S ，不是只枚举大小。

这比直接记录每个子树具体用了哪些编号要清爽很多，因为“互不相同”的限制已经被整体容斥处理掉了。

P4336 [SHOI2016] 黑暗前的幻想乡

有 n 个城市，需要修 $n - 1$ 条路使所有城市连通。

同时有 $n - 1$ 个建筑公司，每家公司给出自己能修的边集。要求：

- ▶ 选出的边构成一棵生成树；
- ▶ 每条边分配给一个能修它的公司；
- ▶ 每个公司恰好负责一条边。

求方案数，对 $10^9 + 7$ 取模。

$$2 \leq n \leq 17.$$

P4336 [SHOI2016] 黑暗前的幻想乡

生成树计数可以用矩阵树定理解决。

对一张带边权无向图，构造 Laplacian 矩阵 L ：

- ▶ $L_{u,u}$ 为与 u 相邻边权之和；
- ▶ $L_{u,v}$ 为 u, v 之间边权之和的相反数。

删除任意一行一列后，剩下矩阵的行列式就是所有生成树的边权乘积之和。

因此如果两点之间有多条可选边，可以直接把它们合成一条权值等于条数的边。

P4336 [SHOI2016] 黑暗前的幻想乡

现在只剩下去重问题：如何保证每个公司都修了一条边？

令公司集合为 C ，其中

$$|C| = n - 1.$$

对一个公司子集 $S \subseteq C$ ，令 $F(S)$ 表示只允许 S 中的公司修路时的方案数，不要求每个公司都被用到。

如果一条城市边能被 S 中 w 个公司修，那么就在矩阵树定理中给这条边权值 w 。

这样矩阵树定理算出的 $F(S)$ ，已经包含了边分配给公司的方案数。

P4336 [SHOI2016] 黑暗前的幻想乡

答案要求所有公司都至少出现一次。

因为生成树正好有 $n - 1$ 条边，公司也正好有 $n - 1$ 个，所以“每个公司至少出现一次”等价于“每个公司恰好出现一次”。

对公司集合做子集容斥：

$$ans = \sum_{S \subseteq C} (-1)^{|C|-|S|} F(S).$$

这一步和前面的“小星星”很像：先允许元素重复或缺失，再容斥出每个元素都出现的方案。

P4336 [SHOI2016] 黑暗前的幻想乡

如果按子集大小分组，也可以写成

$$ans = \sum_{i=0}^{|C|} (-1)^{|C|-i} \sum_{\substack{S \subseteq C \\ |S|=i}} F(S).$$

题解中常把 $|C| = n - 1$ 重新记作 n ，于是形式上就是

$$\sum_{i=0}^n (-1)^{n-i} f(i).$$

也就是说，对每个公司子集，只需要知道它诱导出的带权图的生成树总权值。

P4336 [SHOI2016] 黑暗前的幻想乡

这道题的结构可以概括为：

公司是否出现 $\implies$ 子集容斥 $\implies$ 带权生成树计数.

矩阵树定理负责把生成树的选择和边分配一起统计，容斥负责保证每家公司都真正出现。

Min-Max 容斥

Min-Max 容斥也叫最值反演。它解决的问题是：

已知一些子集的最小值信息，反过来求最大值；或者已知最大值信息，反过来求最小值。

例如两个数时有

$$\max(a, b) = a + b - \min(a, b).$$

三个数时有

$$\begin{aligned} \max(a, b, c) = & a + b + c - \min(a, b) - \min(a, c) - \min(b, c) \\ & + \min(a, b, c). \end{aligned}$$

这已经很像容斥了。

Min-Max 容斥

对非空集合 S , 记

$$\text{Max}(S), \quad \text{Min}(S)$$

分别表示 S 中元素的最大值和最小值。

则有

$$\text{Max}(S) = \sum_{\emptyset \neq T \subseteq S} (-1)^{|T|-1} \text{Min}(T).$$

对所有元素取相反数, 也可以得到对偶形式:

$$\text{Min}(S) = \sum_{\emptyset \neq T \subseteq S} (-1)^{|T|-1} \text{Max}(T).$$

Min-Max 容斥

为什么这个式子成立？

将 S 中元素从大到小排列为

$$x_1 \geq x_2 \geq \cdots \geq x_m.$$

在右侧求和中, x_i 成为 $\text{Min}(T)$, 需要 T 包含 x_i , 并且其它被选元素只能来自 $x_1, \dots, x_{i-1}$ 。

因此 x_i 的总系数为

$$\sum_{j=0}^{i-1} (-1)^j \binom{i-1}{j}.$$

当 $i=1$ 时系数为 1; 当 $i>1$ 时系数为 0。

所以最后只剩下最大值 x_1 。

Min-Max 容斥

还可以推广到第 k 大值。

记 $\text{kMax}(S)$ 表示 S 中第 k 大的元素，则

$$\text{kMax}(S) = \sum_{\substack{T \subseteq S \\ |T| \geq k}} (-1)^{|T|-k} \binom{|T|-1}{k-1} \text{Min}(T).$$

当 $k = 1$ 时，

$$\binom{|T|-1}{0} = 1,$$

就退化回普通的 Min-Max 容斥。

Min-Max 容斥

它最常见的应用场景是“所有元素都出现的期望时间”。

设 t_i 表示第 i 个元素第一次出现的时间。若想知道集合 S 中所有元素都出现的时间，就是

$$\text{Max}\{t_i : i \in S\}.$$

由 Min-Max 容斥，

$$E[\text{Max}(S)] = \sum_{\emptyset \neq T \subseteq S} (-1)^{|T|-1} E[\text{Min}(T)].$$

这里 $\text{Min}(T)$ 表示 T 中至少一个元素出现的时间，通常比“全部出现”容易算。

Min-Max 容斥

例如在离散时间中，若每一步出现 T 中至少一个元素的概率为 p_T ，且每一步相互独立，则

$$E[\text{Min}(T)] = \frac{1}{p_T}.$$

于是困难的“全部出现时间”可以转成若干个“至少一个出现时间”的线性组合。

使用时要注意两点：

- ▶ 枚举的是具体子集 T ，不是只枚举大小；
- ▶ 能直接放进期望，是因为期望具有线性性。

P3175 [HAOI2015] 按位或

初始数字为 0。每次随机选择一个 $[0, 2^n - 1]$ 中的数 A ，出现概率为 p_A ，并将当前数字变成

$$x \leftarrow x \text{ or } A.$$

求使 $x = 2^n - 1$ 的期望操作次数。

若永远无法达到，则输出 INF。要求误差不超过 10^{-6} 。

$$n \leq 20, \quad 0 \leq p_A \leq 1, \quad \sum_A p_A = 1.$$

P3175 [HAOI2015] 按位或

把 n 个二进制位看作 n 个元素。设 t_i 表示第 i 位第一次变成 1 的时间。

目标时间就是

$$\max_{i=1}^n t_i.$$

这正是 Min-Max 容斥的典型形式。

P3175 [HAOI2015] 按位或

记全集 U 为所有二进制位。对非空集合 $S \subseteq U$ ，由 Min-Max 容斥：

$$E \left[\max_{i \in U} t_i \right] = \sum_{\emptyset \neq S \subseteq U} (-1)^{|S|-1} E \left[\min_{i \in S} t_i \right].$$

现在只要求

$$E \left[\min_{i \in S} t_i \right].$$

它表示 S 中至少有一位第一次变成 1 的时间。

P3175 [HAOI2015] 按位或

一次操作能让 S 中至少一位变成 1，当且仅当随机到的数 A 与 S 有公共的 1 位：

$$A \cap S \neq \emptyset.$$

因此成功概率为

$$q_S = \sum_{A \cap S \neq \emptyset} p_A.$$

每次操作独立，所以

$$E \left[\min_{i \in S} t_i \right] = \frac{1}{q_S}.$$

代回可得答案：

$$ans = \sum_{\emptyset \neq S \subseteq U} (-1)^{|S|-1} \frac{1}{q_S}.$$

P3175 [HAOI2015] 按位或

直接计算每个 q_S 仍然较慢。考虑反面：

$$q_S = 1 - \sum_{A \cap S = \emptyset} p_A.$$

与 S 没有公共 1 位，等价于

$$A \subseteq U \setminus S.$$

于是令

$$F(T) = \sum_{A \subseteq T} p_A,$$

则

$$q_S = 1 - F(U \setminus S).$$

P3175 [HAOI2015] 按位或

所以最终公式可以写成

$$ans = \sum_{\emptyset \neq S \subseteq U} (-1)^{|S|-1} \frac{1}{1 - F(U \setminus S)}.$$

其中 $F(T)$ 表示随机数完全落在 T 这些位里的概率总和。

如果存在某一位永远不可能被随机到的数覆盖，那么答案不存在，输出 INF。

等价地，若存在非空 S 使得

$$q_S = 0,$$

则无法完成目标。

因此整题的关键就是先求“完全避开某些位”的概率，再由它得到“至少覆盖一位”的概率。

点减边容斥

在树上计数时，有时一个合法方案并不是对应某一个点，而是对应一个非空连通块。

如果直接枚举“公共点”，同一个方案会被它对应连通块中的每个点重复计算。

但树上的非空连通块 C 满足

$$|V(C)| - |E(C)| = 1.$$

因此可以用

$$\sum_{\text{点 } u} \#\{\text{钦定经过 } u\} - \sum_{\text{边 } e} \#\{\text{钦定经过 } e\}$$

来让每个非空连通块恰好贡献 1。

点减边容斥

这个技巧常用于“若干个树上连通集合存在公共元素”的计数。

设一个方案对应的公共部分为

$$I = \bigcap_i C_i,$$

其中每个 C_i 都是树上的连通集合。

若 $I = \emptyset$ ，则它不会被任何点或边计入；若 $I \neq \emptyset$ ，则 I 仍然是一个连通块。

于是该方案在点侧贡献 $|V(I)|$ ，在边侧贡献 $|E(I)|$ ，相减后贡献

$$|V(I)| - |E(I)| = 1.$$

这就是“点减边”的本质。

P10064 [SNOI2024] 公交线路

给定一棵 n 个点的树，选择若干条树上简单路径作为公交线路。

对于新图 G ，若存在一条被选路径同时经过 u, v ，就在 u, v 之间连边。

要求新图中任意两点距离不超过 2，求合法路径子集数量，对 998244353 取模。

$$1 \leq n \leq 3000.$$

P10064 [SNOI2024] 公交线路

一个关键化简是：只需要考虑叶子。

对每个叶子 u ，记

S_u contains v iff there is a path through u and v

$$S_u = \{v : \text{dist}_G(u, v) = 1\}.$$

原条件等价于：任意两个叶子 u, v ，都有

$$S_u \cap S_v \neq \emptyset.$$

P10064 [SNOI2024] 公交线路

对叶子 u , S_u 是树上的一个连通块。

树上的连通块满足 Helly 性质:

若若干个连通块两两相交, 则它们有公共元素。

因此合法条件可以改写成

$$\bigcap_{u \text{ 为叶子}} S_u \neq \emptyset.$$

而这个公共部分仍然是一个连通块。

记 A_x, A_e 分别表示钦定公共部分经过点 x 或边 e 的方案数。

于是可以套点减边容斥:

$$ans = \sum_x A_x - \sum_e A_e,$$

P10064 [SNOI2024] 公交线路

先考虑如何计算 A_x ，也就是钦定所有叶子的 S_u 都经过点 x 。

以 x 为根。对叶子做容斥，钦定若干叶子不能与 x 连通。

设 f_i 表示当前已经钦定了 i 个叶子不能与 x 连通时的方案数。

合并两个已经处理好的部分 u, v 。设

$$siz_u, siz_v$$

表示两部分的大小， lef_v 表示 v 部分中的叶子数。

若在 v 部分再钦定 j 个叶子断开，则转移为

$$tf_{i+j} \leftarrow f_i \binom{lef_v}{j} 2^{(siz_u-i)(siz_v-j)}.$$

P10064 [SNOI2024] 公交线路

这个转移的含义是：

- ▶ 从 v 部分的叶子中选出 j 个，钦定它们不能与 x 连通；
- ▶ 没有被钦定断开的点之间，跨两个部分的路径可以任意选；
- ▶ 因此出现因子 $2^{(siz_u-i)(siz_v-j)}$ 。

合并完所有儿子后，用容斥还原：

$$A_x = \sum_i (-1)^i f_i.$$

对边 e 的 A_e 也类似：把这条边看作必须被公共连通块经过，再做同样的叶子容斥。

P10064 [SNOI2024] 公交线路

朴素做法是：

- ▶ 对每个点 x 做一次树上背包，求 A_x ；
- ▶ 对每条边 e 做一次类似 DP，求 A_e 。

这说明点贡献和边贡献都可以用同一类“叶子是否断开”的容斥来理解。

真正需要抓住的是：当前公共点或公共边一旦钦定，树就被切成若干部分，每个叶子只关心自己是否还能连到公共部分。

P10064 [SNOI2024] 公交线路

整道题的核心结构可以概括为：

叶子邻域连通块 $\implies$ 公共连通块 $\implies$ 点减边容斥.

互不相等容斥

考虑这样一类计数问题：

有 n 个变量 $a_1, a_2, \dots, a_n$ ，每个 a_i 有自己的可选集合 S_i ，要求

$$a_i \in S_i, \quad a_i \neq a_j \ (i \neq j).$$

直接处理“两两不相等”通常很麻烦。

反过来考虑相等关系：在点集 $\{1, 2, \dots, n\}$ 上建图，若钦定

$$a_i = a_j,$$

就连一条边 (i, j) 。

那么一个连通块内部的所有变量都会被强制相等。

互不相等容斥

用容斥排除所有相等事件。

对边集 E , 记 $C(E)$ 表示图 (V, E) 的连通块划分。

设 $F(P)$ 表示: 按照划分 P , 每个块内部变量都相等时的方案数。

则答案为

$$ans = \sum_E (-1)^{|E|} F(C(E)).$$

按连通块划分 P 合并同类项:

$$ans = \sum_P F(P) \sum_{C(E)=P} (-1)^{|E|}.$$

关键变成计算后面这个容斥系数。

互不相等容斥

对于一个大小为 k 的块, 记

$$g_k = \sum_{\substack{E \\ (V,E) \text{ 连通}}} (-1)^{|E|}.$$

任意边集的带符号和为

$$(1 - 1)^{\binom{k}{2}},$$

因此 $k \geq 2$ 时为 0。

枚举 1 号点所在连通块, 就能得到

$$0 = g_k + (k-1)g_{k-1} \quad (k > 1).$$

结合 $g_1 = 1$, 可得

$$g_k = (-1)^{k-1} (k-1)!.$$

互不相等容斥

因为不同连通块之间互相独立，所以划分 P 的系数为

$$\prod_{B \in P} (-1)^{|B|-1} (|B| - 1)!.$$

最终得到公式：

$$ans = \sum_P F(P) \prod_{B \in P} (-1)^{|B|-1} (|B| - 1)!.$$

使用时的步骤通常是：

- ▶ 先想清楚“若某些变量被强制相等”，方案数 $F(P)$ 怎么算；
- ▶ 再把每个块乘上系数 $(-1)^{|B|-1} (|B| - 1)!$ ；
- ▶ 最后对所有划分求和，或者用集合幂级数/状压 DP 加速。

AT_abc236_h Distinct Multiples

给定 n, m , 求有多少个序列 $a_1, a_2, \dots, a_n$ 满足:

- ▶ $1 \leq a_i \leq m$;
- ▶ $i \mid a_i$;
- ▶ a_i 两两不同。

其中

$$n \leq 16, \quad m \leq 10^{18}.$$

AT_abc236_h Distinct Multiples

题目中的难点正是

$$a_i \neq a_j \quad (i \neq j).$$

因此可以直接套互不相等容斥。

AT_abc236_h Distinct Multiples

先看如果强制一个点集 T 内的变量全部相等，会发生什么。

若

$$a_i = x \quad (i \in T),$$

那么 x 必须同时被 T 中所有下标整除，即

$$\text{lcm}(T) \mid x.$$

所以这个块的取值方案数为

$$\left\lfloor \frac{m}{\text{lcm}(T)} \right\rfloor.$$

这里记

$$\text{lcm}(T) = \text{lcm}\{i : i \in T\}.$$

AT_abc236_h Distinct Multiples

由互不相等容斥，一个大小为 $|T|$ 的相等块还要乘上连通块系数：

$$f(|T|) = (-1)^{|T|-1}(|T|-1)!.$$

因此若把 $\{1, 2, \dots, n\}$ 划分为若干块

$$P = \{T_1, T_2, \dots, T_k\},$$

该划分的贡献为

$$\prod_{T \in P} \left\lfloor \frac{m}{\text{lcm}(T)} \right\rfloor f(|T|).$$

答案就是对所有集合划分求和。

AT_abc236_h Distinct Multiples

为了对所有划分求和，使用状压 DP。

设 dp_S 表示已经确定了点集 S 中所有点的连通块归属后的贡献和。

每次找到最小的未确定点：

$$r = \min(\bar{S}).$$

枚举 r 所在的块 T ，要求

$$T \cap S = \emptyset, \quad r \in T.$$

转移为

$$dp_{S \cup T} \leftarrow dp_{S \cup T} + dp_S \left\lfloor \frac{m}{\text{lcm}(T)} \right\rfloor f(|T|).$$

AT_abc236_h Distinct Multiples

为什么要钦定 r 是最小的未确定点？

因为一个划分中的块没有顺序。如果直接任意枚举下一个块，同一个划分会被重复统计。

固定每次选择“最小未确定点所在的块”，就能让每个划分恰好被生成一次。

对每个非空集合 T ，需要知道两个量：

$$\text{lcm}(T), \quad f(|T|).$$

若 $\text{lcm}(T) > m$ ，则这个相等块没有任何可选取值。

AT_abc236_h Distinct Multiples

连通块系数满足：

$$f(1) = 1, \quad f(k) = -(k-1)f(k-1).$$

最后答案就是所有集合划分的贡献之和。状压 DP 只是为了不重不漏地枚举这些划分。

目录

概率与期望

二项式定理与容斥原理

线性代数相关知识

数域、向量与矩阵

线性代数里的运算通常在一个数域 K 中进行，例如实数、有理数，或者模质数意义下的整数。

长度为 n 的向量可以写成

$$x = (x_1, x_2, \dots, x_n)^T.$$

一个 $n \times m$ 矩阵是一个矩形数表：

$$A = \begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1m} \\ a_{21} & a_{22} & \cdots & a_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nm} \end{pmatrix}.$$

线性组合

若有若干个向量 $v_1, v_2, \dots, v_k$ ，形如

$$c_1 v_1 + c_2 v_2 + \dots + c_k v_k$$

的向量称为它们的线性组合。

所有线性组合构成的集合称为张成空间。

很多问题其实是在问：

- ▶ 某个向量能不能由一组向量线性组合得到；
- ▶ 一组向量中有多少个“真正独立”的方向；
- ▶ 一个线性方程组是否存在解、是否存在唯一解。

线性相关与秩

若存在不全为 0 的系数 c_i ，使得

$$c_1 v_1 + c_2 v_2 + \cdots + c_k v_k = 0,$$

则称这些向量线性相关；否则称为线性无关。

一组向量中最多能选出多少个线性无关的向量，这个数量称为秩。

对矩阵来说：

- ▶ 行秩：行向量张成空间的维数；
- ▶ 列秩：列向量张成空间的维数；
- ▶ 两者总是相等，统一称为矩阵的秩。

线性变换

矩阵可以看作一个线性变换。

若 A 是 $n \times m$ 矩阵, x 是长度为 m 的列向量, 则

$$y = Ax$$

是长度为 n 的列向量。

它满足线性性:

$$A(u + v) = Au + Av, \quad A(cu) = cAu.$$

这就是为什么线性递推、线性 DP 转移都能写成矩阵乘法。

线性方程组的矩阵形式

方程组

$$\begin{cases} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1m}x_m = b_1, \\ a_{21}x_1 + a_{22}x_2 + \cdots + a_{2m}x_m = b_2, \\ \cdots \\ a_{n1}x_1 + a_{n2}x_2 + \cdots + a_{nm}x_m = b_n \end{cases}$$

可以写成

$$Ax = b.$$

高斯消元就是通过等价变形，把这个方程组化成容易求解的形式。

矩阵乘法

若 A 是 $n \times m$ 矩阵, B 是 $m \times p$ 矩阵, 则 $C = AB$ 是 $n \times p$ 矩阵。

第 i 行第 j 列的元素为

$$c_{ij} = \sum_{k=1}^m a_{ik} b_{kj}.$$

直观理解:

- ▶ C 的第 i 行第 j 列, 是 A 的第 i 行与 B 的第 j 列的点积;
- ▶ AB 表示先做 B 对应的线性变换, 再做 A 对应的线性变换。

矩阵乘法的性质

矩阵乘法满足：

$$A(BC) = (AB)C,$$

$$A(B + C) = AB + AC, \quad (A + B)C = AC + BC.$$

但一般不满足交换律：

$$AB \neq BA.$$

n 阶单位矩阵 I 满足

$$AI = IA = A.$$

普通矩阵乘法的复杂度是 $O(nmp)$ ，方阵相乘通常写作 $O(n^3)$ 。

矩阵快速幂

若需要计算

$$A^k = \underbrace{AA \cdots A}_{k \uparrow},$$

可以像普通快速幂一样使用二进制拆分。

初始令

$$ans = I, \quad base = A.$$

当 $k > 0$ 时:

- ▶ 若 k 的最低位为 1, 则 $ans \leftarrow ans \cdot base$;
- ▶ 令 $base \leftarrow base \cdot base$;
- ▶ 令 $k \leftarrow \lfloor k/2 \rfloor$ 。

矩阵快速幂的复杂度

对 n 阶方阵，普通矩阵乘法一次是 $O(n^3)$ 。

矩阵快速幂需要 $O(\log k)$ 次矩阵乘法，因此总复杂度为

$$O(n^3 \log k).$$

典型应用：把线性递推写成矩阵形式。

例如 Fibonacci 数：

$$\begin{pmatrix} F_{i+1} \\ F_i \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} F_i \\ F_{i-1} \end{pmatrix}.$$

从递推到矩阵

若有线性递推

$$f_i = c_1 f_{i-1} + c_2 f_{i-2} + \cdots + c_m f_{i-m},$$

可以令状态向量为

$$S_i = (f_i, f_{i-1}, \dots, f_{i-m+1})^T.$$

则

$$S_i = TS_{i-1},$$

其中

$$T = \begin{pmatrix} c_1 & c_2 & \cdots & c_{m-1} & c_m \\ 1 & 0 & \cdots & 0 & 0 \\ 0 & 1 & \cdots & 0 & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & \cdots & 1 & 0 \end{pmatrix}.$$

行列式

行列式是定义在方阵上的一个数，记作

$$\det(A) \quad \text{或} \quad |A|.$$

对 2×2 矩阵，有

$$\begin{vmatrix} a & b \\ c & d \end{vmatrix} = ad - bc.$$

行列式可以理解为线性变换对“有向体积”的缩放倍数。

若 $\det(A) = 0$ ，说明这个变换把空间压扁了，矩阵不可逆，方程组可能无解或有无穷多解。

行列式的性质

行列式关于行具有如下性质：

- ▶ 交换两行，行列式变号；
- ▶ 某一行乘以 c ，行列式也乘以 c ；
- ▶ 某一行加上另一行的 c 倍，行列式不变；
- ▶ 若有两行相同或线性相关，则行列式为 0。

对列也有完全相同的性质。

此外还有

$$\det(AB) = \det(A) \det(B).$$

行列式与排列

n 阶行列式也可以按排列展开：

$$\det(A) = \sum_{\pi} \operatorname{sgn}(\pi) \prod_{i=1}^n a_{i,\pi_i}.$$

其中 π 枚举 $1, 2, \dots, n$ 的所有排列， $\operatorname{sgn}(\pi)$ 表示排列的奇偶性。

这个定义说明了：

- ▶ 每一项从每行、每列各选一个元素；
- ▶ 奇排列贡献为负，偶排列贡献为正；
- ▶ 直接展开需要 $O(n!)$ ，实际求值通常用高斯消元。

初等行变换

高斯消元只使用三类行变换：

- ▶ 交换两行；
- ▶ 某一行乘以非零常数；
- ▶ 某一行加上另一行的若干倍。

对方程组来说，这些变换不会改变解集。

对行列式来说，它们的影响分别是：

- ▶ 交换两行：行列式变号；
- ▶ 某一行乘以 c ：行列式乘以 c ；
- ▶ 某一行加上另一行的倍数：行列式不变。

高斯消元

高斯消元的目标是把矩阵化成阶梯形。

对每一列 col ，寻找一个尚未使用的非零主元：

$$a_{row,col} \neq 0.$$

找到后：

- ▶ 把这一行交换到当前主元行；
- ▶ 用它消掉下面所有行在这一列的元素；
- ▶ 当前主元行下移一行。

最后非零主元的数量就是矩阵的秩。

高斯消元的消元公式

假设当前主元为 $a_{i,i}$ ，要消掉第 j 行的 $a_{j,i}$ 。

在普通数域或模质数意义下，可以令

$$t = \frac{a_{j,i}}{a_{i,i}},$$

然后对所有 k 执行

$$a_{j,k} \leftarrow a_{j,k} - ta_{i,k}.$$

在模 p 意义下，除以 $a_{i,i}$ 等价于乘它的逆元：

$$t = a_{j,i} \cdot a_{i,i}^{-1} \pmod{p}.$$

高斯消元解方程

解线性方程组时，通常把 $Ax = b$ 写成增广矩阵：

$$(A \mid b).$$

消元后根据秩判断解的情况：

- ▶ 若出现 $0 = c$ 且 $c \neq 0$ ，则无解；
- ▶ 若主元数量等于未知数个数，则有唯一解；
- ▶ 若主元数量小于未知数个数，且没有矛盾行，则有无穷多解。

唯一解时，可以回代求出每个未知数。

高斯消元解方程：回代

消元得到上三角形形式后，最后一行通常只含较少未知量。

从最后一个主元行开始，倒序计算：

$$x_i = \frac{b_i - \sum_{j=i+1}^n a_{ij}x_j}{a_{ii}}.$$

如果在实数上运算，要注意浮点误差，判断 0 时应使用 ε 。

如果在模意义下运算，要求主元存在逆元；最常见的是模质数，此时所有非零数都有逆元。

Gauss-Jordan 消元

另一种常用写法是 Gauss-Jordan 消元：找到主元后，把这一列除主元行外的所有元素都消成 0。

这样最后得到的不是上三角矩阵，而是更接近单位矩阵的形式。

若存在唯一解，最终可以化为

$$(I \mid x),$$

右侧直接就是答案。

它写起来判断方便，但常数比只向下消元再回代稍大。

高斯消元求行列式

对 n 阶方阵求行列式时，只需要把矩阵消成上三角矩阵。

上三角矩阵的行列式等于对角线元素乘积：

$$\det(A) = \prod_{i=1}^n a_{ii}.$$

消元过程中要记录行变换对行列式的影响：

- ▶ 每交换一次行，答案乘以 -1 ；
- ▶ 使用“某行减去另一行的倍数”不会改变行列式；
- ▶ 若某一行找不到非零主元，则行列式为 0 。

矩阵树定理

矩阵树定理用于计算无向图的生成树数量。

对一张带边权无向图，令边 (u, v) 的权值为 $w(u, v)$ 。构造 Laplacian 矩阵 L ：

$$L_{ii} = \sum_{(i,j) \in E} w(i,j),$$

$$L_{ij} = -w(i,j) \quad (i \neq j).$$

如果 i, j 之间没有边，则 $w(i, j) = 0$ 。

矩阵树定理

设 $L^{(r)}$ 表示删去 L 的第 r 行和第 r 列后得到的矩阵。

矩阵树定理说明：

$$\det L^{(r)} = \sum_T \prod_{e \in T} w(e),$$

其中 T 枚举原图的所有生成树。

特别地，若所有边权都是 1，则

$$\det L^{(r)}$$

就是生成树个数。

为什么要删一行一列

Laplacian 矩阵每一行的元素和都是 0:

$$\sum_j L_{ij} = 0.$$

因此所有行向量线性相关，必有

$$\det L = 0.$$

对连通图来说， L 的秩恰好是 $n - 1$ 。

删去任意一行一列后得到的余子式，刚好提取出生成树的总贡献。选哪一行哪一列不影响结果。

矩阵树定理的使用

使用时只需要做三件事：

- ▶ 建立 $n \times n$ 的 Laplacian 矩阵；
- ▶ 任意删去一行一列，得到 $(n-1) \times (n-1)$ 矩阵；
- ▶ 用高斯消元求这个矩阵的行列式。

如果题目要求对模数取模，就把整个行列式放在对应的模意义下理解。

矩阵树定理的例子

三角形图 1, 2, 3, 每条边权为 1。

Laplacian 矩阵为

$$L = \begin{pmatrix} 2 & -1 & -1 \\ -1 & 2 & -1 \\ -1 & -1 & 2 \end{pmatrix}.$$

删去第 3 行第 3 列, 得到

$$\begin{pmatrix} 2 & -1 \\ -1 & 2 \end{pmatrix},$$

行列式为 3, 正好对应三角形的 3 棵生成树。

有向图版本

矩阵树定理也有有向图版本，用来统计以某个点为根的有向生成树。

若统计所有边都指向根 r 的内向树，可以构造：

$$L_{ii} = \sum_{(j,i) \in E} w(j,i),$$

$$L_{ij} = -w(j,i) \quad (i \neq j).$$

也就是说，对角线记录入边权值和，非对角线记录从 j 指向 i 的边。

删除根 r 对应的行列后求行列式，即为以 r 为根的内向树总权值。

LGV 引理

LGV 引理用于统计 DAG 上的若干条点不相交路径。

给定起点 $A_1, A_2, \dots, A_k$ 和终点 $B_1, B_2, \dots, B_k$ 。

令

$$M_{ij} = \sum_{P: A_i \rightarrow B_j} w(P),$$

即 M_{ij} 是从 A_i 到 B_j 的所有路径权值和。

其中一条路径的权值通常定义为边权乘积；若无权，则每条路径权值为 1。

LGV 引理

LGV 引理说明:

$$\det(M) = \sum_{\pi} \operatorname{sgn}(\pi) \sum_{\mathcal{P}} \prod_{i=1}^k w(P_i).$$

其中 $\mathcal{P}$ 枚举所有满足

$$P_i : A_i \rightarrow B_{\pi_i}$$

且两两点不相交的路径组。

行列式中的相交路径组会通过符号反转互相抵消，最后只留下不相交路径组。

LGV 引理的常见形式

很多平面图或网格图中，起点和终点的顺序会强制：

A_i 只能与 B_i 配对.

此时其他排列对应的不相交路径组不存在，于是

$$\det(M)$$

就直接等于从 A_i 到 B_i 的 k 条两两不相交路径的总权值。

这就是 LGV 引理在计数题里最常见的用法。

LGV 引理的使用流程

使用时先把问题拆成两个层次：

- ▶ 确定起点 A_i 和终点 B_j ;
- ▶ 计算每个 $A_i \rightarrow B_j$ 的路径总数或总权值;
- ▶ 把这些值放进矩阵 M ;
- ▶ 求 $\det(M)$ 。

前半部分负责普通路径计数，后半部分负责用行列式自动排除相交路径组。

LGV 引理的例子

在网格图上，只允许向右或向上走。

从 (x_1, y_1) 到 (x_2, y_2) 的路径数为

$$\binom{(x_2 - x_1) + (y_2 - y_1)}{x_2 - x_1},$$

若 $x_2 < x_1$ 或 $y_2 < y_1$ ，则路径数为 0。

因此若要数若干条互不相交的单调路径，只要把这些组合数填入 M_{ij} ，再求

$$\det(M).$$

LGV 引理为什么会抵消

行列式展开时，每一项选择一个排列：

$$\prod_i M_{i, \pi_i}.$$

展开 M_{i, π_i} 后，相当于枚举一组从 A_i 到 B_{π_i} 的路径。

如果路径组有交点，取第一个相交点，把两条路径在这个点之后的后缀交换。

这样会改变终点配对排列的奇偶性，但不会改变总权值。因此相交路径组成对抵消。

CF348D Turtles

给定一个 $n \times m$ 网格，有些格子是障碍。

两只乌龟都从 $(1, 1)$ 出发，到达 (n, m) ，每一步只能走到

$$(x + 1, y) \quad \text{或} \quad (x, y + 1).$$

要求两条路径除了起点和终点以外不能经过同一个格子。

求合法路径对数量，对

$$10^9 + 7$$

取模。

$$2 \leq n, m \leq 3000.$$

CF348D Turtles

直接看两条从 $(1, 1)$ 到 (n, m) 的路径，它们一定会在起点和终点相交。

但 LGV 引理要求的是点不相交路径组。

因此把公共起点和公共终点“剥掉”：

$$A_1 = (1, 2), \quad A_2 = (2, 1),$$

$$B_1 = (n - 1, m), \quad B_2 = (n, m - 1).$$

之后只需要统计从 A_1, A_2 到 B_1, B_2 的两条点不相交路径。

CF348D Turtles

从 $(1, 1)$ 出发后的第一步只有两种:

$$(1, 2), \quad (2, 1).$$

到达 (n, m) 前的最后一步也只有两种:

$$(n-1, m), \quad (n, m-1).$$

若两条路径除起终点外不相交, 那么它们必须分别占用这两个第一步位置, 也必须分别占用这两个最后一步位置。

于是原问题等价于两个源点、两个终点的不相交路径计数。

CF348D Turtles

在单调网格中， $(1, 2)$ 在 $(2, 1)$ 的“上方”， $(n - 1, m)$ 也在 $(n, m - 1)$ 的“上方”。

如果令

$$(2, 1) \rightarrow (n - 1, m),$$

那么另一条路径从 $(1, 2)$ 到 $(n, m - 1)$ ，两条单调路径必然交叉。

因此唯一可能的不交匹配是

$$(1, 2) \rightarrow (n - 1, m),$$

$$(2, 1) \rightarrow (n, m - 1).$$

CF348D Turtles

令 $ways(S, T)$ 表示从格子 S 到格子 T , 只向下或向右走, 且不经过障碍的路径数。

固定起点 S 后做一次 DP:

$$dp_{x,y} = \begin{cases} 0, & (x,y) \text{ 是障碍,} \\ dp_{x-1,y} + dp_{x,y-1}, & \text{否则.} \end{cases}$$

初始 $dp_S = 1$ 。若某个位置在 S 的左上方, 或者越界, 贡献自然视为 0。

本题只需要从两个源点各跑一次 DP。

CF348D Turtles

建立 2×2 矩阵

$$M_{ij} = \text{ways}(A_i, B_j).$$

由 LGV 引理，所有不交路径组的带符号贡献为

$$\det(M) = M_{11}M_{22} - M_{12}M_{21}.$$

因为交叉匹配不可能产生不交路径组，所以这个行列式正好等于答案。

因此答案就是

$$\text{ans} = (M_{11}M_{22} - M_{12}M_{21}) \bmod (10^9 + 7).$$

CF348D Turtles

若 $(1, 2), (2, 1), (n - 1, m), (n, m - 1)$ 中有障碍或越界，对应路径数会变成 0，DP 可以自然处理。

注意最终答案可能为负，需要调整：

$$ans = (ans + MOD) \bmod MOD.$$

这些边界情况不用单独改变思路：它们只会让矩阵中的某些路径数变成 0。

线性基

线性基通常用于处理“从一个集合中选出若干个数，求它们异或和”的问题。

异或可以看成二进制向量在 $\mathbb{F}_2$ 上的加法：

$$1 + 1 = 0, \quad 1 + 0 = 1.$$

因此，每个非负整数都可以看成一个 0/1 向量。

子集异或和，就是这些向量在 $\mathbb{F}_2$ 上的线性组合。

异或张成

设 S 是一个整数集合。

从 S 中任选若干个数，把它们异或起来，所有可能得到的值构成 S 的张成，记作

$$\text{span}(S).$$

也就是说：

$$x \in \text{span}(S)$$

当且仅当 x 可以表示成 S 中若干个数的异或和。

线性基关心的就是如何用尽量少的数，保留同样的张成。

线性相关与线性无关

若集合 S 中存在一个元素 x , 满足

$$x \in \text{span}(S \setminus \{x\}),$$

则称 S 线性相关。

直观地说, 就是 x 可以被其它元素异或出来。

这时删去 x 不会改变张成:

$$\text{span}(S) = \text{span}(S \setminus \{x\}).$$

若不存在这样的元素, 则称 S 线性无关。

线性基的定义

若集合 B 满足：

- ▶ $\text{span}(B) = \text{span}(S)$;
- ▶ B 线性无关;

则称 B 是 S 的一个线性基。

线性基保留了原集合所有可能的子集异或和，但删掉了所有可以由其它元素表示的冗余元素。

因此它可以看成异或空间中的一组“独立方向”。

线性基的性质

线性基有几个重要性质：

- ▶ 它与原集合张成相同；
- ▶ 它内部没有非空子集的异或和为 0；
- ▶ 线性基中的元素不能再被其它基元素表示；
- ▶ 若线性基大小为 r ，则可以表示出 2^r 个不同的异或值。

最后一条来自线性无关性：每个基元素选或不选，对应的异或和都不同。

构造线性基

构造线性基的思想与高斯消元非常像。

对于一个新数 x ，从高位到低位看它的最高 1：

- ▶ 若当前最高位已经有基元素，就用这个基元素把这一位消掉；
- ▶ 若当前最高位还没有基元素，就把 x 作为这一位的代表加入线性基。

如果最后 x 被消成 0，说明它已经能由现有线性基表示，不会带来新的方向。

最高位代表

常见的线性基会让每个基元素拥有不同的最高位。

例如，若 b_i 表示最高位为 i 的基元素，则

$$b_i$$

负责消去其它数的第 i 位。

这和高斯消元中的主元很像：

- ▶ 最高位对应主元列；
- ▶ 基元素对应主元行；
- ▶ 插入新数就是一次消元过程。

判定能否表示

判断一个数 x 是否属于 $\text{span}(S)$ ，可以用线性基消元。

从高位到低位尝试消去 x 的最高 1：

- ▶ 若这一位有基元素，就异或它，把这一位消掉；
- ▶ 若这一位没有基元素，则无法继续消去。

若最终可以消成 0，则

$$x \in \text{span}(S).$$

否则 x 不能由原集合中的若干元素异或得到。

最大异或和

线性基可以用来求集合的最大子集异或和。

设当前答案为 ans 。从高位到低位考虑基元素，如果

$$ans \oplus b_i > ans,$$

就把 b_i 异或进答案。

直观上，这是贪心地让更高位尽量变成 1。

由于线性基保留了所有子集异或和，所以在线性基上求到的最大值，就是原集合的最大子集异或和。

线性基的合并

两个集合 A, B 的线性基可以合并。

设 L_A 是 A 的线性基, L_B 是 B 的线性基。

因为

$$\text{span}(A \cup B) = \text{span}(L_A \cup L_B),$$

所以只要把 L_B 中的基元素逐个加入 L_A , 就能得到 $A \cup B$ 的线性基。

这说明线性基不仅能描述一个集合, 也能作为可合并的信息摘要。

P11620 [Ynoi Easy Round 2025] TEST_34

给定一个长度为 n 的序列 $a_1, a_2, \dots, a_n$, 有 m 次操作。

操作分为两类:

- ▶ $opt = 1$: 把区间 $[l, r]$ 内的每个数都异或上 v ;
- ▶ $opt = 2$: 从区间 $[l, r]$ 中任选若干个数, 可以不选。设它们的异或和为 x , 询问

$$x \text{ XOR } v$$

的最大值。

数据范围为

$$1 \leq n, m \leq 5 \times 10^4, \quad 0 \leq a_i, v \leq 10^9.$$

P11620 [Ynoi Easy Round 2025] TEST_34

如果没有修改，区间询问很自然会想到维护线性基：把区间里的数张成的空间取出来，再从 v 开始贪心，让答案尽量变大。

困难在于区间异或修改。线性基支持加入和合并，但没有像前缀和那样的“减法”，所以很难直接给一整段线性基打标记。

因此这题的关键不是先优化线性基，而是先把修改变得更容易维护。

P11620 [Ynoi Easy Round 2025] TEST_34

考虑对异或做差分。设

$$b_1 = a_1, \quad b_i = a_i \text{ xor } a_{i-1} \quad (i \geq 2).$$

若把区间 $[l, r]$ 中的每个 a_i 都异或上 v , 那么差分数组中只有边界会改变:

$$b_l \leftarrow b_l \text{ xor } v,$$

若 $r < n$, 还会有

$$b_{r+1} \leftarrow b_{r+1} \text{ xor } v.$$

于是原来的区间修改, 被转化成了差分数组上的两个单点修改。

P11620 [Ynoi Easy Round 2025] TEST_34

差分之后，还需要把询问中的原数组 a 和差分数组 b 联系起来。

由定义可以得到

$$a_i = b_1 \text{ xor } b_2 \text{ xor } \cdots \text{ xor } b_i.$$

对一个固定区间 $[l, r]$ ，真正想要的是

$$\text{span}\{a_l, a_{l+1}, \dots, a_r\}.$$

下面要证明，它可以换成

$$\text{span}(\{a_l\} \cup \{b_{l+1}, b_{l+2}, \dots, b_r\}).$$

P11620 [Ynoi Easy Round 2025] TEST_34

先看一个方向。对任意 $i > l$, 有

$$a_i = a_l \text{ xor } b_{l+1} \text{ xor } b_{l+2} \text{ xor } \cdots \text{ xor } b_i.$$

所以每个 a_i 都能由

$$\{a_l\} \cup \{b_{l+1}, \dots, b_r\}$$

表示出来。

这说明

$$\text{span}\{a_l, \dots, a_r\} \subseteq \text{span}(\{a_l\} \cup \{b_{l+1}, \dots, b_r\}).$$

P11620 [Ynoi Easy Round 2025] TEST_34

再看另一个方向。对任意 $i \in [l+1, r]$ ，由差分定义可知

$$b_i = a_i \text{ xor } a_{i-1}.$$

因为 a_i 和 a_{i-1} 都在区间 $[l, r]$ 中，所以 b_i 也属于

$$\text{span}\{a_l, a_{l+1}, \dots, a_r\}.$$

同时 a_l 本来就在这个空间里。因此两个空间相等：

$$\text{span}\{a_l, \dots, a_r\} = \text{span}(\{a_l\} \cup \{b_{l+1}, \dots, b_r\}).$$

P11620 [Ynoi Easy Round 2025] TEST_34

现在询问就变清楚了。

若 $l = r$ ，区间里只有一个数，只需要比较选或不选 a_l ：

$$\max(v, v \text{ xor } a_l).$$

若 $l < r$ ，先取出差分数组中

$$b_{l+1}, b_{l+2}, \dots, b_r$$

的线性基，再把 a_l 加进去。得到的就是原区间

$$a_l, a_{l+1}, \dots, a_r$$

的线性基。

最后从 v 开始在线性基上贪心，就能得到 $x \text{ xor } v$ 的最大值。

P11620 [Ynoi Easy Round 2025] TEST_34

因此可以维护差分数组 b 的区间线性基。

区间异或修改只会影响 b_l 和 b_{r+1} ，所以只需要做单点修改；区间询问则提取 $b_{[l+1,r]}$ 的线性基，并补上一个 a_l 。

这里 a_l 可以由差分前缀异或得到：

$$a_l = b_1 \text{ xor } b_2 \text{ xor } \cdots \text{ xor } b_l.$$

整体复杂度为

$$O((N + M) \log N \log^2 V),$$

空间复杂度为

$$O(N \log V).$$

CF1100F Ivan and Burgers

给定一个长度为 n 的序列

$$a_1, a_2, \dots, a_n.$$

有 q 次询问，每次给出区间 $[l, r]$ ，要求从

$$a_l, a_{l+1}, \dots, a_r$$

中任选若干个数，使它们的异或和尽量大。

换句话说，就是求区间 $[l, r]$ 的最大子集异或和。

数据范围为

$$1 \leq n, q \leq 5 \times 10^5, \quad 0 \leq a_i \leq 10^6.$$

CF1100F Ivan and Burgers

这是一道前缀线性基的经典板子题。

如果只问前缀 $[1, r]$ ，我们直接把 $a_1, \dots, a_r$ 依次插入线性基即可。

现在询问的是任意区间 $[l, r]$ 。一个自然想法是：在前缀 $[1, r]$ 的线性基中，只保留真正来自 $[l, r]$ 的信息。

为了做到这一点，线性基里每个基元素除了存值，还要存一个位置。

设

$$p_i$$

表示最高位为 i 的基元素，

$$pos_i$$

表示这个基元素对应的原序列位置。

CF1100F Ivan and Burgers

对前缀 $[1, r]$ 建出这样的线性基之后，如果一个基元素满足

$$pos_i \geq l,$$

它就可以被区间 $[l, r]$ 使用。

因此回答询问 $[l, r]$ 时，只用这些满足 $pos_i \geq l$ 的基元素做贪心，就能求出区间最大子集异或和。

这里真正需要保证的是：每一位上的 pos_i 都尽量靠右。

只有这样，当我们丢掉所有 $pos_i < l$ 的基元素时，剩下的部分仍然能保留区间 $[l, r]$ 的张成信息。

CF1100F Ivan and Burgers

怎样让 pos_i 尽量靠右?

插入一个来自位置 w 的数 x 时, 仍然从高位到低位消元。

若当前最高位 i 还没有基元素, 就把

$$p_i = x, \quad pos_i = w$$

放进去。

若这一位已经有基元素, 但

$$pos_i < w,$$

就让新的数占据这一位, 把原来的 p_i, pos_i 继续往低位消。

这相当于在不改变张成空间的前提下, 把每个主元位置尽量换成更靠右的元素。

CF1100F Ivan and Burgers

这个交换不会破坏线性基。

因为若 x 和原来的 p_i 最高位相同，那么交换后继续令

$$x \leftarrow x \text{ xor } p_i$$

向低位消元，本质上仍然是在同一个线性空间里做基变换。

它只改变了“哪一个元素作为这一位的代表”，没有改变前缀能表示出的所有异或值。

这样处理完前缀 $[1, r]$ 后，每个最高位对应的代表都尽量靠右，于是区间过滤条件 $pos_i \geq l$ 才是可靠的。

CF1100F Ivan and Burgers

整体做法就很清楚了：

按位置从左到右扫描序列，维护当前前缀的线性基；每插入一个 a_r ，就得到前缀 $[1, r]$ 的线性基。

对所有右端点为 r 的询问 $[l, r]$ ，只取当前线性基中满足

$$pos_i \geq l$$

的基元素，从高位到低位贪心更新答案。

因为值域不超过 10^6 ，线性基的位数很小。总复杂度为

$$O((n + q) \log V).$$

谢谢。